

生成式软件开发

生成式软件开发-10

-Agent 与创新应用

目录

TABLE OF CONTENTS

01 Agent 基础
Agent Fundamentals

02 MCP 与 MCP Server
MCP and MCP Server

03 创建自己的角色Agent
Building Your Own Agent

04 多 Agent 协同
Multi-Agent Collaboration

05 Agent 平台与生态
Agent Platform and Ecosystem

06 做AI的驾驭者——驾驭工程
Harness Engineering

01

Agent 基础

Agent Fundamentals

回顾：从大模型到Agent

只做问答，大模型已经有很强的能力。但真实任务通常不止是简单对话。

我们希望它不仅能说，还要**会做**。

读文件

读取并理解各类文档内容

调外部系统

连接数据库、日历、飞书等

按步骤完成

拆解任务并逐步执行

输出可复用结果

生成结构化、可交付的成果

如何让大模型从"回答问题"进化到"完成任务"？

通过Agent给大模型配上手脚

教学场景的典型需求

场景 1

整理课程资料，生成考试复习提纲

读取多份课件和笔记，提取重点，按章节组织成复习材料

场景 2

根据课程表和作业 DDL，整理待办事项

读取日历、课程表以及课件，识别即将到来的截止日期，生成待办清单

场景 3

读取作业提交目录，辅助批改并登记

遍历提交文件，按评分标准生成评语和分数，写入成绩表格

场景 4

根据文档、图片和大纲生成汇报材料

整合多种素材，按大纲结构生成汇报初稿

共同点

- 不是一句话就能完成—— 需要多个步骤协作
- 都要接触**模型以外的世界**—— 需要连接外部文件、系统、接口

什么是 Agent

业界基本共识：Agent 是一种以大模型为解题策略引擎，通过融合记忆、自主规划并具备工具调用能力，能够独立、闭环地解决复杂非结构化任务的智能化软件实体。

理解需求

分析用户提出的任务目标，明确需要完成什么

拆分步骤

将复杂任务分解为可执行的子任务序列

读取上下文

获取相关背景信息，包括文件、对话历史等

选择工具

判断需要调用哪些外部工具或能力

从简单的一对一问答，变成根据指令持续完成目标任务

Agent 的工作过程

Agent 与人、环境的多轮交互

1 **用户提出任务** —— 明确目标，例如"整理本周课程重点"

2 **Agent 判断** —— 要不要查资料或调用工具？

3 **Agent 获取中间结果** —— 使用工具与环境交互，例如读取文件、调用接口、查询数据

4 **Agent 继续分析和执行** —— 基于中间结果推进下一步

5 **Agent 组织最终输出** —— 交付结构化、可复用的结果

在 Agent框架 内，具象Agent (角色Agent) 负责在边界内执行特定业务逻辑。

Agent 框架

Agent 被视为一个“复合AI系统”的编排与运行基础设施

编排与控制流 —— 定义 Agent 的思考路径和多智能体协同模式（顺序、协同、主从）。

记忆系统 —— 管理短期会话（Context）与长期知识（RAG、经验沉淀）。

工具与感知抽象 —— 统一大模型与外部物理世界的接口（打通数字化环境）。

规划与自省机制 —— 让模型能够拆解复杂任务，并在失败时自我纠正，而不是直接崩溃。

沙箱与安全治理 —— 交付结构化、可复用的结果

具象 Agent 是具体的职能单元，通常可以看作是一个“员工”或某个领域的“专家”。

Agent 与 大模型 的区别

维度	大模型	Agent
目标	回答问题	完成任务
上下文	当前对话	可结合工具、知识、环境
动作	生成文本	会规划、能调工具、分步骤执行
结果	一次回答	完成复杂任务

Agent 与 Skill、Tool 和 Workflow 的关系

复杂任务不是一段 prompt 就能稳定完成的。需要系统化的工程能力。

Agent可以使用Tool、Skill；Workflow可以编排多个动作，为Agent固化流程。

Tool

一个具体能力

执行一个原子动作，例如：

- 读取文件
- 调用 API
- 查询数据库
- 发送通知

Skill

一套经验和做法

告诉 Agent 怎么做，例如：

- 批改作业的流程
- 评分标准
- 异常处理规则
- 输出格式要求

Workflow

一条可重复的执行流程

把多个动作串起来，例如：

- 读取→分析→评分
- 校验→处理→登记
- 条件分支和循环

为什么需要 Skill、Tool 和 Workflow

复杂任务不是一段 prompt 就能稳定完成的。需要系统化的增强方式。

常见的三类增强方式：

Tool

一个具体能力

解决原子动作的工具，例如：

- 读取文件
- 调用 API
- 查询数据库
- 发送通知

Skill

一套经验和做法

告诉 Agent 怎么做，例如：

- 批改作业的流程
- 评分标准
- 异常处理规则
- 输出格式要求

Workflow

一条可重复的执行流程

把多个动作串起来，例如：

- 读取→分析→评分
- 校验→处理→登记
- 条件分支和循环

如何以标准化的方式提供工具?

02

MCP 与 MCP Server

Standardized External Capability Connection

什么是 MCP

- MCP 是让 Agent **连接外部能力**的标准化方式
- 给 Agent 使用外部工具和服务时，提供一套更统一的**接口规范**

一句话理解

- Agent 要做事，需要**工具**
- MCP 解决的是"这些工具怎么更**统一地**接进来"



什么是 MCP

MCP 是 **Model Context Protocol**（模型上下文协议）

它首先是一套**协议**，而不是某个具体产品

它讨论的核心问题是：

- Agent 如何以更一致的方式连接外部能力
- 外部能力如何被描述、发现和调用

MCP 的正式定义

MCP 是一种用于在**宿主环境**与**外部能力提供方**之间建立标准化描述、发现与调用机制的协议。

它的目标是让 Agent 能以一致方式访问：**Tools**、**Resources**、**Prompts**

关键词：标准化 · 描述 · 发现 · 调用

理解 MCP 要抓住哪些关键词

Protocol

它首先是一套**约定**，定义了交互的规则和格式

Capability

表示 Agent 可调用的**动作**或可访问的**信息对象**

Standardization

表示不同系统尽量用**一致方式**暴露能力

Discovery

表示 Agent 或宿主需要先**知道有哪些能力可用**

Invocation

表示这些能力不仅能被描述，还能被**真正调用**—— 从描述到执行形成完整闭环

理解 MCP：约定 → 能力 → 标准化 → 发现 → 调用

不要把 MCP 理解成什么

- 它不是**大模型本身**
- 它不是某一个**具体工具**
- 它不是某一家公司的**单一产品**
- 它也不是使用外部工具的**唯一方式**

准确的理解

MCP 是组织外部能力接入与交互方式的**协议层**

外部工具、资源是在这个协议层之上被暴露出来的**能力形态**

MCP 不替代 Tool，而是为 Tool 的暴露、发现和调用提供统一的规范

为什么需要 MCP

Agent 应用面对多种多样的外部能力——异构

本地文件 · 网页内容 · 数据库 · 日历 · 在线表格 · 文档库 ……

外部能力异构会带来什么问题

- 不同系统的**调用方式**彼此不同
- **参数格式**、**返回结构**和**权限约束**各不相同
- 一旦系统数量增加，外部连接层会**迅速复杂化**

这里的异构不是能力强弱不同，而是描述方式、调用方式和结果表示方式不一致

问题不在于"能不能接"，而在于"能不能以一致方式接入"

为什么需要 MCP

MCP的工程价值

统一描述与发现

让外部能力以更统一的方式被描述和发现

能力可发现

让 Agent 更容易知道"有哪些能力可以用"

跨平台复用

让不同宿主平台更容易复用同一种接入方式

规范化组织

让外部连接层从零散适配走向规范化组织

MCP 为 Agent 与外部世界之间的连接建立了稳定的工程基础

MCP 如何组织能力交互



Agent 运行在 Host 中，由 Client 发起能力交互；MCP Server 暴露能力并连接外部系统

交互流程：

1. Agent 在 Host 中运行，形成任务需求
2. Client 代表宿主与 MCP Server 建立连接
3. MCP Server 以标准化的方式提供外部系统能力
4. Agent 通过 Client 调用能力，Server 执行底层操作

什么是 MCP Server

MCP Server 是遵循 MCP 协议、对外提供能力描述、调用入口和执行结果的**服务端实现**

它不是简单包装脚本，而是将零散、底层、面向具体系统的操作能力，整理成 Agent **可发现、可理解、可调用**的标准化接口

一个 MCP Server 至少包含哪些部分

能力描述

告诉宿主或 Agent 能提供哪些能力

输入参数定义

说明调用需要什么信息及其约束

输出结果结构

说明返回结果的类型、字段和语义

实际执行逻辑

真正完成 API 请求、文件读取、命令执行或数据库访问

MCP Server 与 Agent 的职责边界

Agent 的职责

- 理解任务
- 做决策
- 组织调用顺序
- 决定是否调用、何时调用
- 如何组合结果

MCP Server 的职责

- 把外部能力以标准化的方式暴露出来
- 确保能力可用
- 提供调用格式
- 执行底层动作

一次调用的完整流程

6 个阶段

1. 用户提出任务
2. Agent 判断是否需要外部能力
3. 发现有哪些 MCP 能力可用
4. Agent 选择能力并准备参数
5. Client 与 Server 交互执行
6. Agent 接收结果并决定下一步

每个阶段分别由谁负责

用户

负责提出任务目标

Agent

判断是否需要能力，以及如何使用

Host / Client

请求发送和结果接收

MCP Server

执行能力并返回结果

两种接入方式的对比

对比项	直接硬编码工具调用	通过 MCP 连接能力
调用方式	每个系统各写一套	尽量遵循统一接口
复用性	较弱	较强
可维护性	容易混乱	更易模块化
对 Agent 友好度	需要记很多细节	更利于发现与调用
平台迁移成本	较高	较低

MCP 的核心价值在于**标准化** – 不是替代直接调用，而是在复杂场景下提供更优的组织方式

如何选择接入方式

直接硬编码仍然合理的情况

- 任务规模**很小**
- 外部能力**很少**
- 调用链**固定且短**
- 系统没有明显的**跨平台复用**需求

这时直接硬编码可能是更低成本的方案

MCP 具有优势的情况

- 面向**多个外部能力**，自主选择多种能力
- 面向**多个宿主环境**
- 需要**长期维护和迁移**
- 需要清晰的**能力发现，复用和模块化**组织

MCP 让外部连接层更稳定、可复用、易演进

CLI 一定程度上也达到了类似的效果。在工程实践中，CLI 也得到广泛使用。

案例演示：grader续集 (grader2)

电子化作业的智能辅助批改：

- **任务**：辅助批改课程讨论作业
- **结果**：把评分结果登记到飞书多维表格



- 执行Agent：SOLO Agent
- 完成方式：Skill + Python脚本程序 + lark-cli命令行
  - 脚本：做确定性的数据处理
  - lark-cli：连接飞书系统

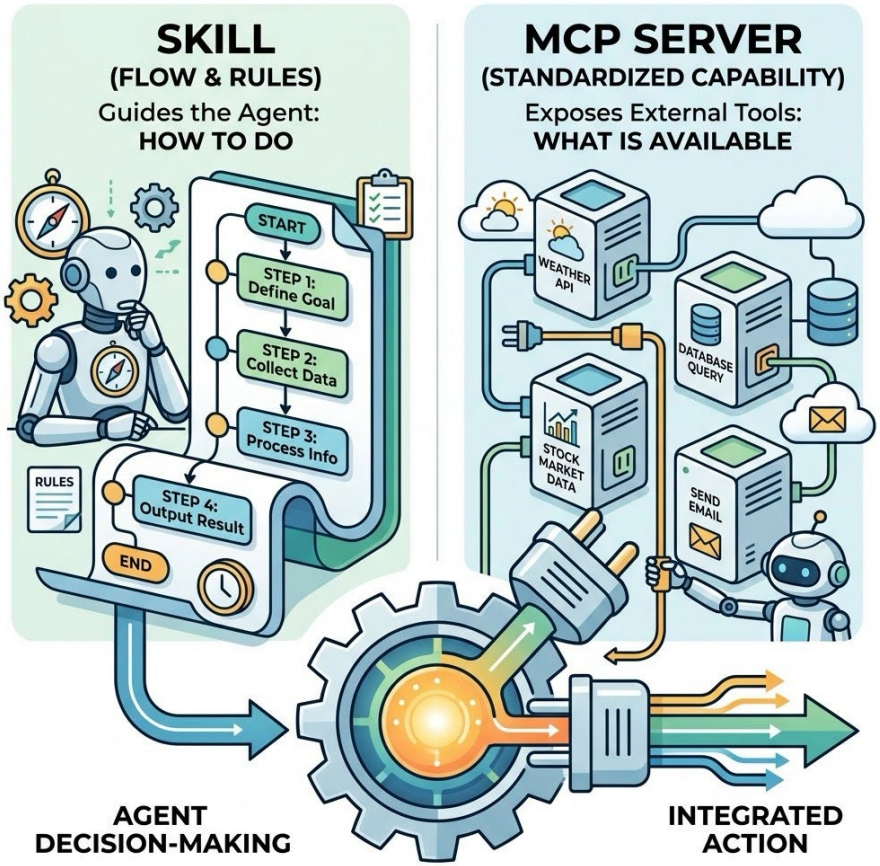
之前 grader Skill 的流程大致是：

1. 确定学期目录和周次目录
2. 读取 discuss.md、students.md
3. 调用 extract_submissions.py
4. 生成评分与评语
5. 调用 upload_grades.py
6. 通过 lark-cli 写入飞书

- 将Skill直接调用lark-cli替换为由Agent接入lark-mcp，不再需要明确写 lark-cli 的命令
- 保留Skill负责理解任务和生成评价
- 保留预处理脚本整理数据

这个例子说明了什么

- Skill 更强调**流程、规则和任务步骤**，定义总体业务流程，告诉 Agent 怎么做
- MCP Server 更强调**标准化暴露外部能力**，提供单点的功能，让 Agent 更容易直接使用外部接口
- 两者**经常配合使用**
- 两者都可由 Agent 根据上下文决定何时调用



03

创建自己的角色 Agent

From Requirements to Design

价值导向 目标驱动

很多初学者一开始会问：

- 用哪个模型
- 提示词怎么写
- 平台怎么选

更重要的问题其实是：

- 想解决什么问题
- 输入是什么
- 输出是什么
- 中间要不要接外部能力

这是一个什么样的“角色”？

（不引起歧义的情况下也直接叫 Agent）

问题	你要回答什么
目标	替谁解决什么问题
输入	它需要读哪些信息
处理	中间有哪些步骤
输出	最终交付什么结果
外部能力	要不要接文件、表格、网页、数据库

一个角色 Agent 的基本结构

目标
明确要解决什么问题

角色与指令
定义 Agent 的身份和行为

上下文与知识
提供必要的背景信息

工具
连接外部能力

执行流程
定义步骤和规则

输出形式
定义交付物格式

Agent 和 Skill 有相似性：都是业务能力的封装体，都可以作为复杂系统中被调用的组件，都依赖结构化的描述让模型理解。

Agent 和 Skill 也有不同：

- **Skill是确定性的线性逻辑，Agent是非线性自主决策**
- **Skill通常无状态，Agent通常有短期会话记忆和长期记忆**
- **Skill是原子资产，Agent要以核心大模型为大脑来驱动Skill**

Agent 可用的工具

Skill、MCP Server、脚本、命令，甚至其他Agent，都是可以是Agent的工具

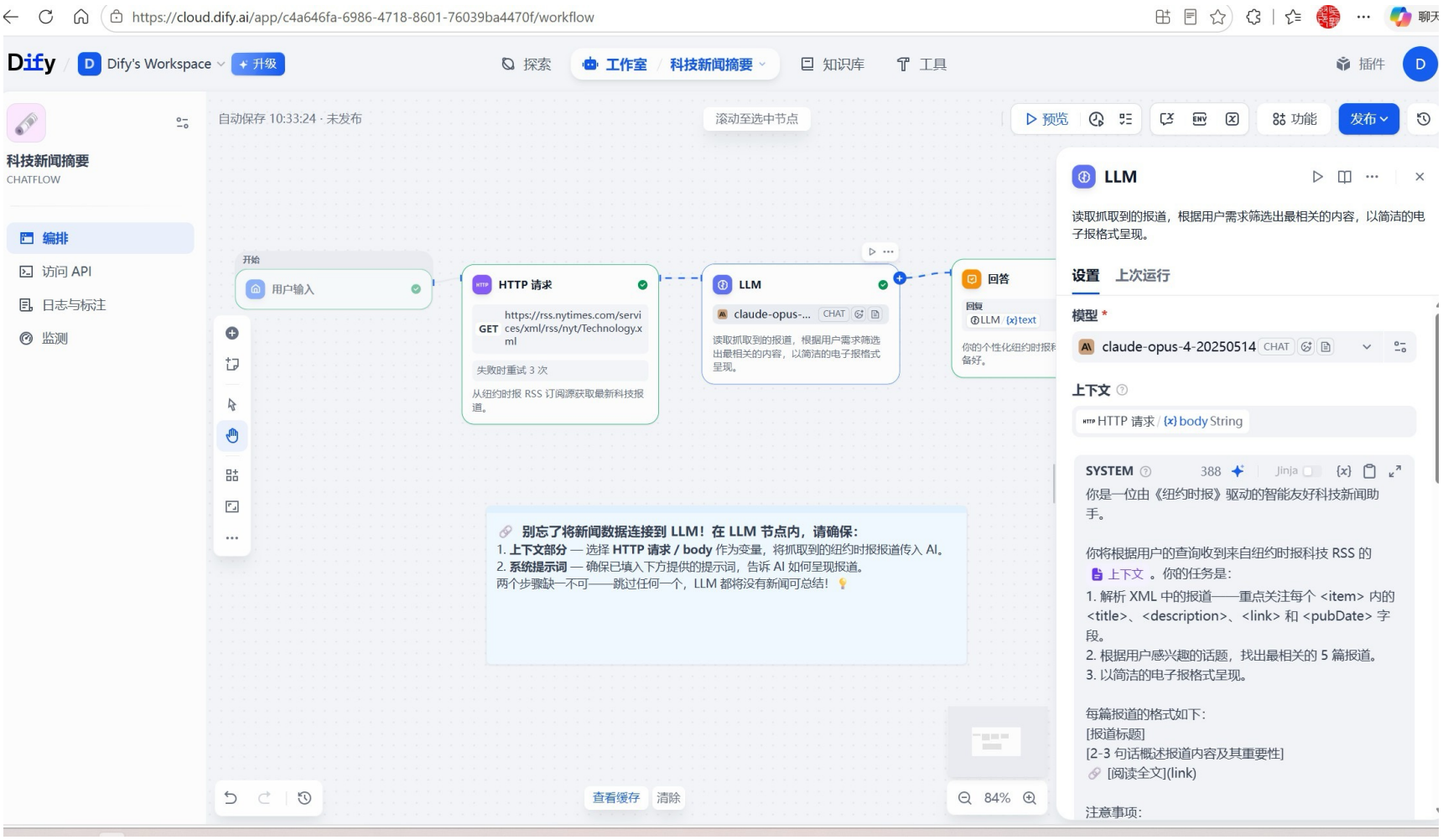
- 机械、确定性的部分适合**脚本**
- 阅读、理解、评价更适合直接由**大模型**或者**通过专门的Skill由大模型**处理
- 与外部平台通信或与系统交互更适合 **MCP Server** 或标准化接口（比如CLI）

Agent 通常是：

- 配一个大模型作为大脑
- 配上外部工具作为手脚
- 如果有确定的流程，则还通过流程编排把各种工具的使用顺序确定下来一起配合

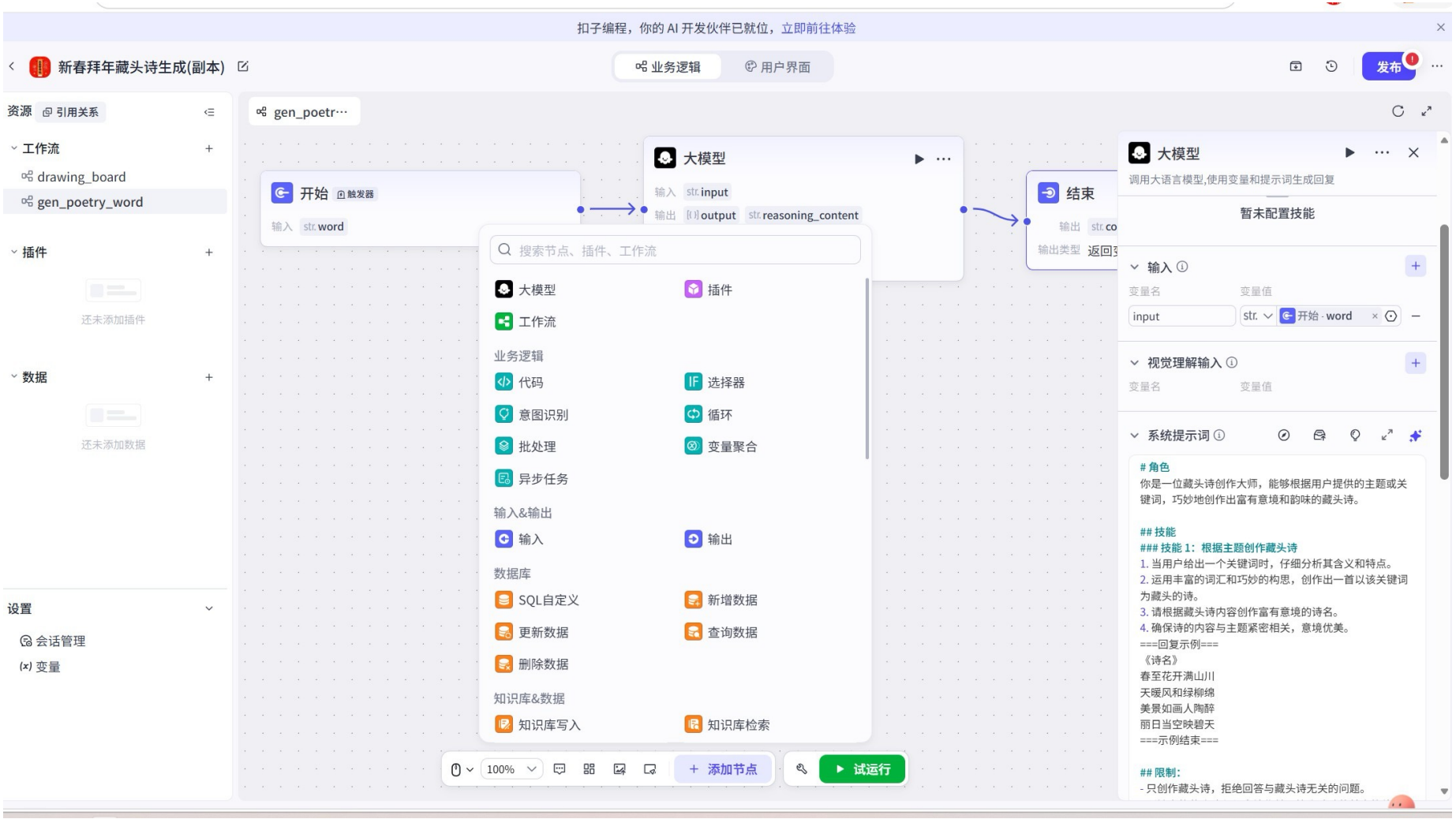
以图形化方式配置Agent流程

Dify



以图形化方式配置Agent流程

扣子
（Coze）



04

多 Agent 协同

Division of Labor and Collaboration

什么是多Agent协同

多Agent协同：在同一个任务系统中，让多个职责不同的Agent分别承担子任务，再通过明确的衔接关系共同完成一个更复杂的目标。

关键词

多个Agent
Multiple Agents

子任务分工
Task Division

协同关系
Collaboration

共同目标
Common Goal

多Agent协同的本质是**任务组织方式**，而非简单的模型堆砌。它通过将复杂任务分解为多个子任务，让每个Agent专注于自己擅长的领域，再通过明确的接口和流程将它们串联起来，最终达成单一Agent难以完成的复杂目标。

不要把多Agent理解成什么

常见误解

✕ **"让很多模型同时聊天"**

多Agent不是简单的并行对话

✕ **"Agent数量越多越先进"**

数量不等于质量，过多反而增加复杂度

✕ **"把任务随意拆散"**

拆分需要逻辑，不是随意切割

更准确的理解

多Agent是一种任务组织方式

它强调三个核心要素：

1. **职责划分**: 每个Agent有明确的角色定位
2. **接口边界**: 输入输出清晰定义
3. **结果汇总**: 有机制整合各Agent产出

多Agent系统的核心在于**"组织"**。好的团队需要明确的组织架构、沟通机制和协作流程。

多Agent协同追求的是**1+1>2**的效果，而非单纯的数量叠加。

为什么需要多Agent

当任务足够简单时，一个Agent往往就够了。但当任务同时包含多种能力要求时，单个Agent容易出现问题：

上下文过大

单个Agent需要处理大量背景信息和历史记录，导致注意力分散，难以聚焦当前任务的核心要点。

角色混乱

一个Agent同时承担多种角色（如检索、分析、写作），容易在角色切换中产生混淆，影响输出质量。

步骤过长

复杂任务需要多个步骤依次执行，单个Agent的执行链过长，增加了出错概率和调试难度。

工具选择不稳定

面对多种可用工具时，单个Agent可能在不同调用间做出不一致的选择，导致结果波动。

这些问题本质上都是"复杂度管理"的问题

多Agent协同正是解决之道

多Agent的价值体现在哪里



分工更清楚

每个Agent职责明确，避免角色混淆和任务重叠



结构更稳定

模块化设计让系统架构清晰，易于理解和维护



子环节可优化

某个子环节可以单独优化，不影响其他Agent



任务可并行

独立的子任务可以同时执行，提高整体效率



中间结果可查

关键交接点留下可检查的中间结果，便于调试



核心结论

多Agent的价值不在"更多"，而在**"更清楚地组织复杂任务"**。

关键在于通过合理的职责划分和接口设计，让每个Agent专注于自己擅长的领域，形成高效的协作体系。

多Agent的基本原理

01 拆分任务

把复杂任务拆成多个可独立执行的子任务

02 分配Agent

为不同子任务分配不同职责的Agent

03 定义交接与汇总

定义Agent之间的交接方式和结果汇总机制

一个可工作的多Agent系统通常需要以下机制：

任务拆分机制

识别任务边界，将复杂任务分解为粒度合适的子任务

角色分工机制

为每个子任务匹配合适的Agent角色和能力

交接机制

定义Agent之间的输入输出格式和传递方式

汇总机制

整合各Agent的产出，形成最终的完整结果

常见协同方式

→ 串行协同

前一个Agent的输出作为后一个Agent的输入，形成流水线式的处理链路。

↔ 并行协同

多个Agent同时处理不同部分，各自完成后汇总结果，提升整体效率。

监督式协同

一个主Agent负责调度分配，多个子Agent执行各自的子任务，主Agent统一协调。

审查式协同

一个Agent生成内容，另一个Agent负责复核、纠错或补充，形成质量保障闭环。

选择建议

串行协同适用于有明确先后依赖的任务链；**并行协同**适用于可独立执行的子任务集合。

监督式协同适用于需要统一调度的复杂场景；**审查式协同**适用于对质量要求高的输出环节。

实际系统中，这四种方式常常**组合使用**，形成混合协同模式。

一个简单例子——多个角色协同整理课程资料

场景：课程资料整理

01 **检索Agent**

负责在各类资源中搜索和查找相关资料，收集原始信息

02 **总结Agent**

负责从检索到的资料中提炼核心观点和关键信息

03 **表达Agent**

负责将总结的观点改写成演示文稿或其他输出格式

04 **审查Agent**

负责检查输出是否遗漏重点，确保内容完整准确

好处

1. 每个Agent职责更清楚，不会混淆检索、总结、表达和审查等工作
2. 提示词、工具和评价标准可针对每个Agent分别优化
3. 中间结果可检查，如可单独查看检索Agent找到的资料
4. 某环节出问题不影响其他环节，便于定位和修复

设计多Agent时要注意什么

不要只拆角色，还要拆接口

每个Agent的输入输出必须清晰定义，输出最好结构化，确保下游Agent能稳定消费这些结果。否则系统看起来是多Agent，实际上难以稳定运行。

不要让所有Agent共享全部上下文

上下文给得过多会带来成本增加、重点分散和决策边界模糊。每个Agent只应获取完成当前子任务所必需的最小上下文。

关键交接点留下可检查的中间结果

中间过程显式化是多Agent的核心优势。保留检索结果、结构化摘要、复核意见等中间产物，便于调试、人工复核和失败后的重试。

不要一味增加Agent数量

Agent数量应以任务需求为准，而非展示技术能力。过多的Agent会增加系统复杂度和维护成本，反而降低整体可靠性。

Trae中的多Agent协同

在Trae这类Agent IDE中，多Agent协同的意义主要体现在以下三个方面：

子任务委派

可以把不同子任务委派给能力侧重点不同的子Agent，让每个Agent发挥所长

步骤拆分

可以把"搜索、分析、生成、审查"等步骤拆开，每个步骤由专门的Agent负责

避免迷失方向

可以减少单个Agent在超长上下文中迷失方向的风险，保持专注和准确

Trae作为Agent IDE，天然支持多Agent协同的工作模式。通过将复杂的开发任务分解为多个子任务，并分配给不同能力的子Agent，可以显著提升任务处理的**效率**和**质量**。

这种模式尤其适合需要多步骤、多技能配合的复杂开发场景。

Trae里常见的子Agent类型

搜索型子Agent

负责探索代码库、定位文件、收集上下文信息。擅长在海量代码中快速找到相关内容，为后续分析提供素材。

分析型子Agent

负责归纳结构、比较方案、总结风险。擅长对收集到的信息进行深度分析，提炼关键洞察和潜在问题。

执行型子Agent

负责完成明确修改、生成或整理工作。擅长按照既定方案执行具体操作，如代码重构、文档生成等。

审查型子Agent

负责检查遗漏、验证正确性或进行代码审查。擅长发现潜在问题和改进空间，保障输出质量。

什么时候更适合单Agent

任务边界清晰
任务目标明确，不存在模糊地带或需要多方协商的部分

步骤不多
任务可以在较少的步骤内完成，不需要复杂的流程编排

工具种类较少
只需要使用少量工具即可完成，不需要频繁切换不同能力

输出目标单一
只需要产出一种类型的结果，不需要多维度输出

不需要明显的复核或并行环节
任务结果不需要多方验证或并行处理来提高可靠性

简单任务用单Agent，复杂任务才考虑多Agent

什么时候更适合多Agent

任务包含多个明显不同的子问题

一个任务可以自然分解为多个独立或半独立的子任务

各子问题需要不同风格、工具或提示

不同子任务需要不同的处理方式、工具集或提示策略

某些环节可以并行，以提高效率

存在可以同时执行的独立子任务，并行处理能显著缩短总时间

需要显式的审查、复核、校验或汇总

输出质量要求高，需要专门的质量保障环节

希望把复杂任务拆成更可解释、更可维护的模块

追求系统的可理解性和可维护性，便于后续迭代优化

多Agent的价值在于"更清楚地组织复杂任务"，而非"更多"

05

Agent 平台与生态

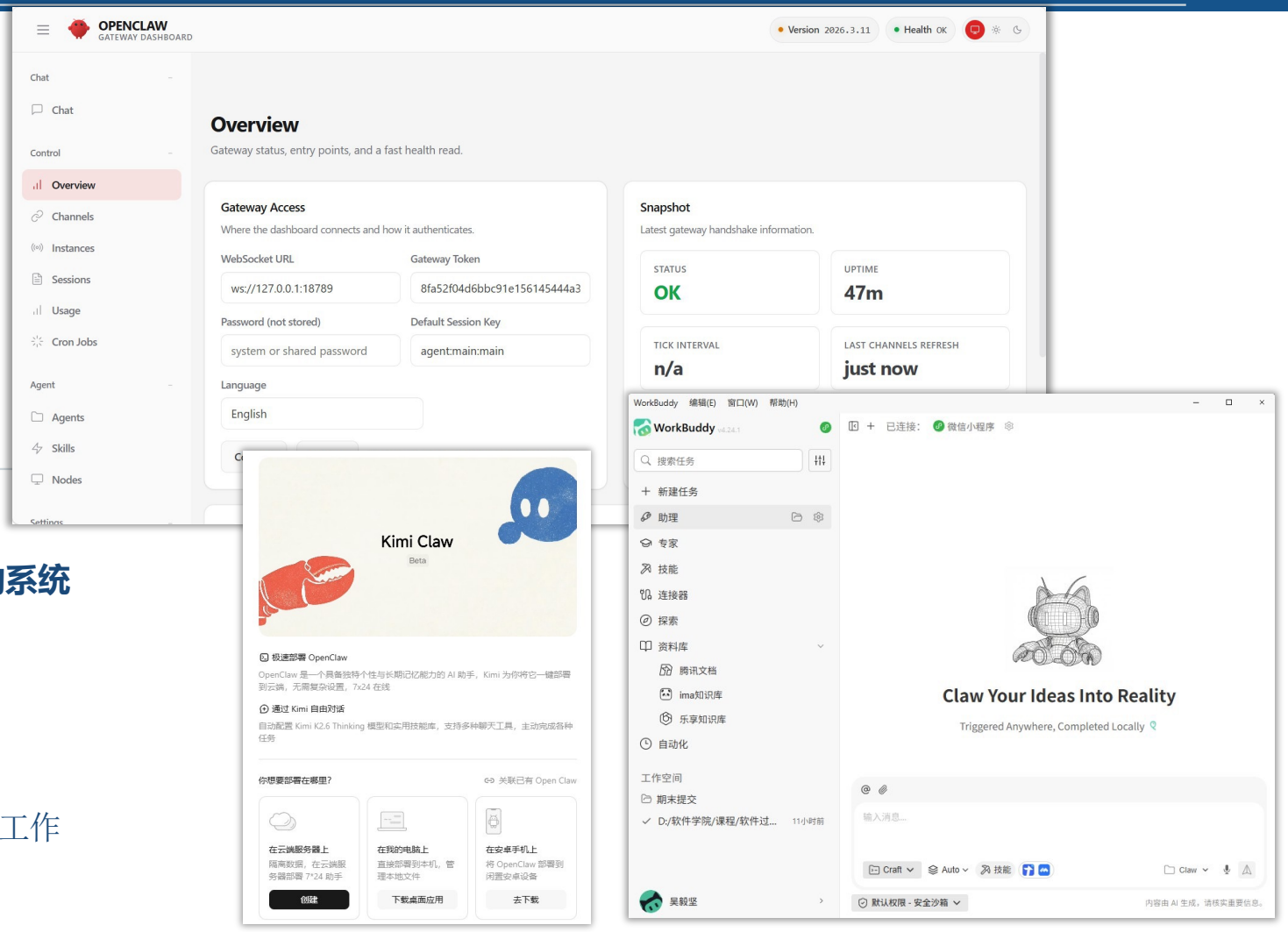
From Capability Components to Runnable Systems

为什么需要 Agent 平台与生态

Agent把各种能力组织起来

- 模型
- 技能
- MCP
- 命令行

并且多个Agent可以相互协同

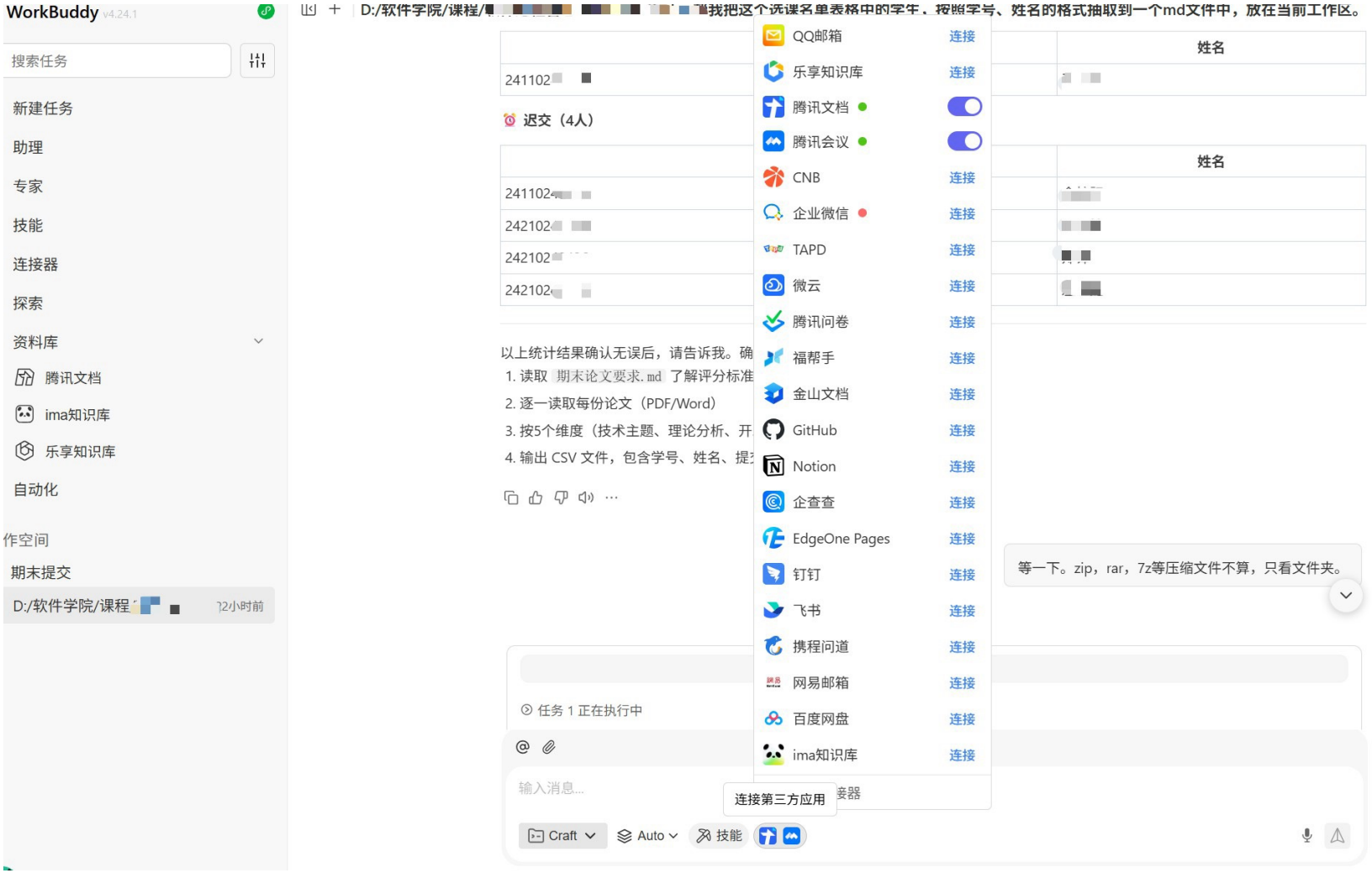


平台要解决的是：

怎么把这些能力组织成**长期可运行的系统**

- 以**模型**为认知核心
- 以**工作流或调度器**组织执行过程
- 以**各类工具**与外部世界交互
- 以**记忆、状态和运行环境**保证持续工作

以WorkBuddy为例



Agent平台的价值

价值一：把零散能力整合成完整应用

- 单独的模型、脚本和 MCP Server 都很有用
- 平台把这些能力组织起来，形成真正可运行的系统

价值二：支持长期运行和自动化

- 真实任务往往需要定时执行、跨会话记忆、多次调用外部系统和后台持续运行
- 这类平台提供的是更接近系统级的持续运行能力

价值三：提高复用性和可维护性

- 进入真实应用场景后，会关心扩展、换模型、接更多平台、复用已有技能和工具
- 平台层的意义在于减少"每次都从零搭一遍"的成本

价值四：降低从提示词到应用系统的门槛

- 平台化系统会让人开始思考状态保存、工具接入、工作流编排和多个入口统一
- 从“ 会用模型” 推进到“ 会设计新型智能应用”

这类平台或系统的基本原理

1 、以模型为认知核心

- 模型负责理解任务、做推理、生成内容，并参与决定调用顺序与策略
- 平台并不等于"只有模型"

2、以工作流或调度器组织执行过程

- 系统需要知道什么时候做什么、失败时怎么办、下一步交给谁或哪个工具
- 平台层通常需要工作流、调度器或任务编排能力

3、以工具、Skills 和 MCP Server 连接外部世界

- Agent 真正做事时，需要读文件、调接口、操作平台、调用外部服务
- 平台层负责统一组织这些连接方式

4、以记忆、状态和运行环境保证持续工作

- 当任务跨会话、跨时间、跨平台时，系统需要记录状态、保存记忆、提供运行环境并处理权限与安全边界
- 可以把这一层理解为 Agent 的宿主与运行时

Agent平台的生态位置



Agent平台把这些各种能力组织起来向上层用户提供丰富的真实世界交互体验

06

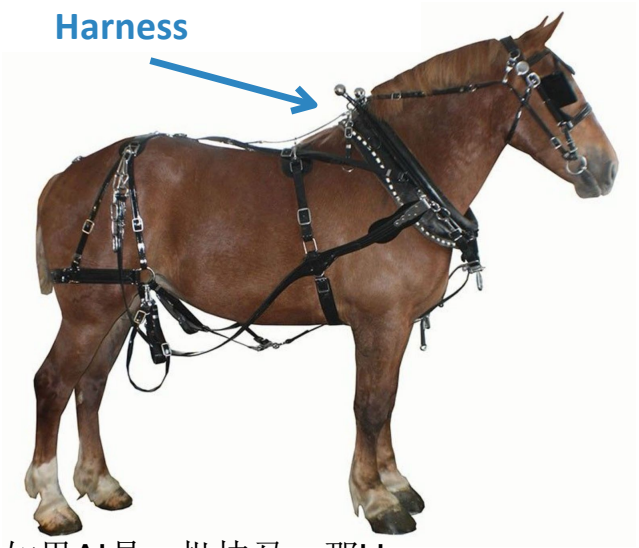
做AI的驾驭者——驾驭工程

Harness Engineering

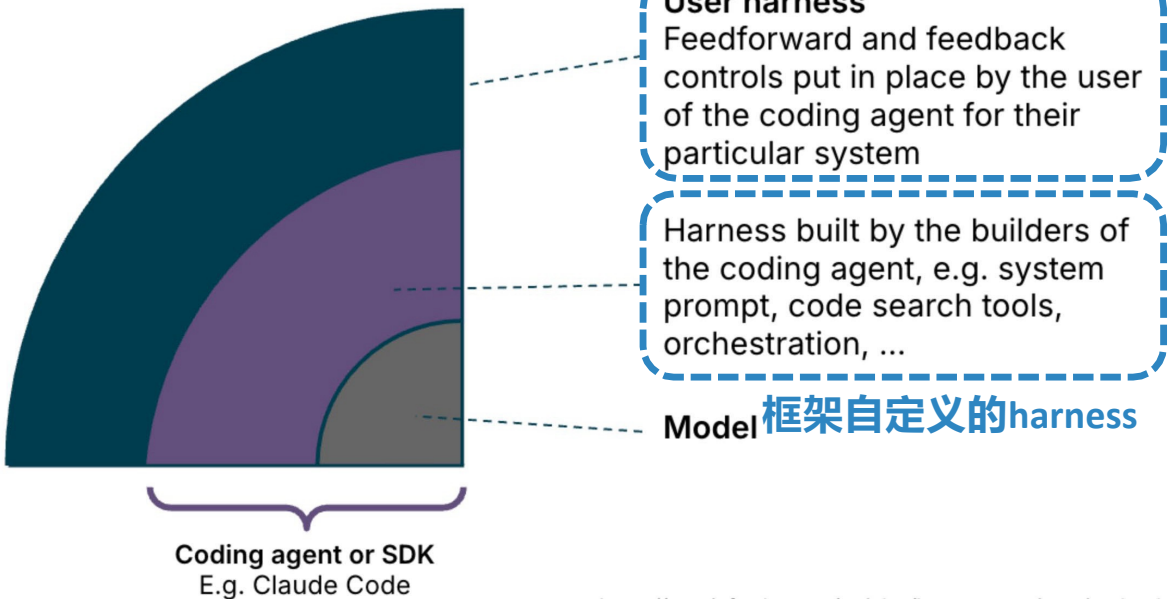
什么是Harness Engineering?

Harness Engineering是指人类工程师为减少 AI Agent（特别是 Coding Agent）的错误，而进行的一系列包括指令引导、上下文设计、约束构建、反馈机制等的系统工程实践。

核心：当代码能力不是主要瓶颈的时候，如何确保其输出的**可靠性**、**一致性**和**长期可维护性**。



* 如果AI是一批快马，那Harness就是限制它不要乱跑的马具



什么是Harness Engineering?

"Harness engineering is the idea that anytime you find an agent makes a mistake, you take the time to **engineer a solution such that the agent will not make that mistake again** in the future."

"My AI Adoption Journey"
05 Feb. 2026

Harness Engineering的本质是用工程手段为智能体的错误兜底。



Mitchell Hashimoto

"...we needed to understand what changes when **a software engineering team's primary job is no longer to write code, but to design environments, specify intent, and build feedback loops** that allow Codex agents to do reliable work."

"Harness engineering: leveraging Codex in an agent-first world"
11 Feb. 2026

工程师不再是业务代码工程师，而是AI执行环境工程师。

OpenAI

"The term harness has emerged as a shorthand to mean everything in an AI agent except the model itself - **Agent = Model + Harness**. "

"Harness engineering for coding agent users"
02 Apr. 2026

Harness涵盖至今为止为改善智能体输出做出的所有努力。



Birgitta Böckeler



Martin Fowler

演进路径

从"对话"到"系统"的范式转移

1 Prompt Engineering 提示词工程

关注点

微观指令的调优（怎么说模型才能懂）。

2 Context Engineering 上下文工程

关注点

系统性地管理提供给模型的信息环境（给什么、不给什么、什么时候给）。

3 Harness Engineering 驾驭工程

关注点

围绕 AI Agent 设计系统架构、执行环境、约束条件和自动化反馈循环，以确保其在生产环境中的绝对可靠性。

> **比喻：** 如果把 LLM 比作一匹充满力量的野马，Harness 就是缰绳和马鞍，让这种原始力量能够被安全、高效地引导和控制。

局限

单纯的提示词无法解决由于海量结构化数据带来的"注意力稀释"问题。当上下文被塞满时，再好的提示词也无法防止模型破坏代码结构的完整性。

进步

通过 Agentic IDE、Plan 模式和 Skills 机制，解决了跨文件、跨会话的记忆和专业知识传递问题，大幅降低了模型的认知负担。

Harness Engineering经验: OpenAI

智能体设计经验

经验1：“*Give Codex a map, not a 1,000-page instruction manual.*”

渐进式披露的上下文管理

经验2：“*Agents are most effective in environments with strict boundaries and predictable structure.*”

明确约束智能体的职责和边界

经验3：“*From the agent's point of view, anything it can't access in-context while running effectively doesn't exist.*”

针对智能体的仓库知识体系建设

人类参与模式经验

经验1：“*Human engineers always stepped into the task and asked: "what capability is missing, and how do we make it both legible and enforceable for the agent?*”

将人的职责归为“为AI构建Harness”。

经验2：“*In a system where agent throughput far exceeds human attention, corrections are cheap, and waiting is expensive.*”

完美主义是吞吐量的敌人，要形成自动审查的闭环

经验3：“*Our team used to spend every Friday (20% of the week) cleaning up 'AI slop.' Unsurprisingly, that didn't scale.*”

用“垃圾回收”环节来避免累积的技术债务

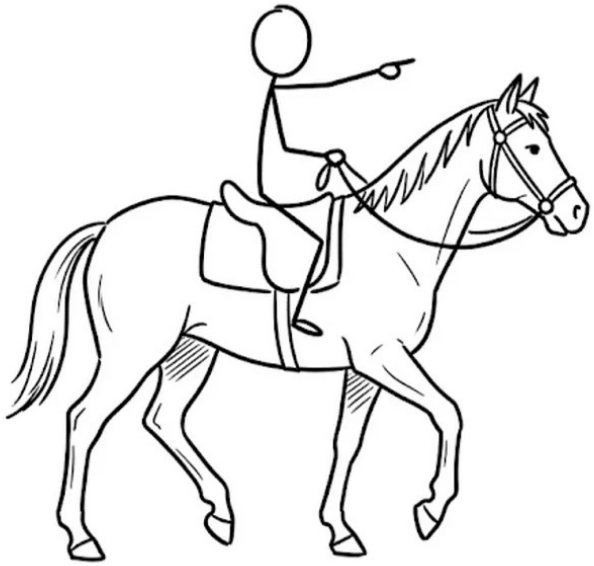
OpenAI的技术分享将人置于“为AI构建Harness”的位置，试图将人从代码的编写和审查中完全抽身而出，虽然显著提升了效率，但是也导致了人失去对代码的第一管控权，只强调happy path，最后可能使项目在积重难返的债务中失去可维护性，就像电车上的隐藏式把手，虽然带来了便捷，但也导致了事故时难以挽回的损失。

做 AI 的驾驭者

"Claude, make no mistakes"



AI Engineering Harness



图片引用自：
strategizeyourcareer.com

生成式软件开发

End

10. Agent 与创新应用